Zephyr Project

IT-Schulungen.com
Technology & Transformation

Praxis 4: Overlay und Diagnose

Fehleranalyse

1.13

Vom Overlay zur korrigierten Anwendung

Fünf Schritte mit bewusst eingebautem Fehler

01

Konfigurieren

Trainingsknoten und Properties im Overlay anlegen

02

Verwenden

DT_PATH und DT_PROP im C-Code einsetzen

03

Nachweisen

zephyr.dts und geändertes Laufzeit-verhalten prüfen

04

Stören

Kleinen Fehler gezielt und reproduzierbar einbauen

05

Diagnostizieren

Symptom, Ebene, Ursache, Prüfung und Korrektur verbinden

Ergebnis: Das Overlay ändert das Verhalten; ein eingebauter Fehler wird systematisch korrigiert.

app.overlay definiert den Trainingsknoten

DATEI

app.overlay

liegt in der Projekt-Root
wird beim Build eingelesen
ergänzt den Board-Baum
bleibt Anwendungseigentum

KNOTEN

/training

unterhalb des Root-Knotens
über DT_PATH erreichbar
einfach im zephyr.dts auffindbar

PROPERTIES

Zwei Werte

interval-ms als Integer
message als String
steuern Wartezeit und Ausgabe

Vor dem Build festlegen: Welche Ausgabe und welches Intervall beweisen die Overlay-Wirkung?

Overlay mit zwei Properties

app.overlay

```
/ {
  training {
    interval-ms = <750>;
    message = "Overlay aktiv";
  };
};
```

01 **Integer**

interval-ms wird als Zellenwert geschrieben und später für K_MSEC verwendet.

02 **String**

message wird als Zeichenkette definiert und für die Konsolenausgabe gelesen.

Nach jeder Overlay-Änderung zuerst im generierten zephyr.dts prüfen, was tatsächlich wirksam ist.

Properties im C-Code verwenden

src/main.c

```
#define TRAINING DT_PATH(training)

BUILD_ASSERT(DT_NODE_EXISTS(TRAINING));

uint32_t ms = DT_PROP(TRAINING, interval_ms);
const char *msg = DT_PROP(TRAINING, message);

printk("%s\n", msg);
k_sleep(K_MSEC(ms));
```

01 Referenz

DT_PATH(training) liefert die compile-time Knotenkennung.

02 Auslesen

DT_PROP liest Integer- und String-Property typisiert aus.

03 Verhalten

Message und Intervall werden in Ausgabe und Wartezeit sichtbar.

Quellcode bleibt gleich aufgebaut; die konkreten Werte kommen aus dem Devicetree.

Die Diagnosekette konsequent anwenden

Symptom	Was ist sichtbar: Compilerfehler, Warnung, fehlendes Artefakt oder falsche Ausgabe?
Ebene	C/C++, CMake, Kconfig, Devicetree, Linker oder Laufzeit zuordnen
Ursache	Erste relevante Meldung mit Dateipfad, Zeilennummer und Kontext lesen
Prüfung	Minimalen Reproduktionsschritt und eine überprüfbare Hypothese festlegen
Korrektur	Kleinste passende Änderung durchführen, neu bauen und Ergebnis nachweisen

Nicht die letzte rote Zeile bekämpfen: Die erste relevante Meldung weist meist näher zur Ursache.

Fehler zuerst einer Ebene zuordnen

ÄNDERUNG UND SYMPTOM VERKNÜPFEN

Die zuletzt geänderte Datei ist ein starker Hinweis – aber noch kein Beweis für die Ursache.

C / C++	Syntax, Typen, Header und falsche Makroverwendung
CMake / Linker	fehlende Quellen, Targets, Bibliotheken oder Symbole
Kconfig	unerfüllte Abhängigkeiten und abweichende wirksame Werte
Devicetree	Syntax, Pfade, Properties, Bindings oder fehlende Knoten
Laufzeit	Prozessausgabe, Hänger, Signal und Exit-Status untersuchen

Die Ebene bestimmt das nächste Werkzeug – nicht die Lautstärke der letzten Fehlermeldung.

Minimal reproduzieren & neu bauen

TERMINAL

```
$ west build
$ grep -n 'training' build/zephyr/zephyr.dts
$ west build -t run
$ echo $?

# bei verändertem Build-Kontext
$ west build -p always -b native_sim .
```

01 Inkrementell

Für lokale C- oder Overlay-Änderungen zuerst den vorhandenen Build nutzen.

02 Pristine

Bei Board-, Fragment- oder Build-Kontextwechsel kontrolliert neu konfigurieren.

Reproduktion notieren: Änderung, exakter Befehl, erste Meldung, Korrektur und neuer Nachweis.

Praxis 4 abgeschlossen

ERFOLG

Overlay wirkt

Knoten nachgewiesen

Properties gelesen

Verhalten geändert

Fehler korrigiert

DIAGNOSE

Kette angewendet

Symptom erfasst

Ebene bestimmt

Ursache geprüft

Korrektur nachgewiesen

AUSBLICK

Weitere Werkzeuge

GDB für Ablauf und Zustand

Valgrind für Speicherfehler

AddressSanitizer

UndefinedBehaviorSanitizer

Lernziel: Symptom → Ebene → Ursache → Prüfung → Korrektur wird auf den eingebauten Fehler angewendet.